

# Task Description

Design and implement a service class (or set of classes) responsible for calculating the positions of **structural supports (mounts)** and **inter-panel joints** for a solar array.

## Input Data

1. **Panel List:** A list of `(x, y)` coordinates representing the **top-left corner** of each panel.
  - *Assumption:* All panels have the same dimensions and orientation.
  - *Panel Dimensions:* `Width = 44.7`, `Height = 71.1` (units).
2. **Rafter Configuration:**
  - Rafters are vertical lines spaced **16 units** apart.
  - You can decide what is the `x` coordinate of the first rafter.

## Business Logic & Constraints

### 1. Support Calculation (Mounts)

Supports attach the panel to the rafters. Calculate coordinates for supports based on the following rules:

- **Rafter Alignment:** Supports can *only* be placed where a rafter exists.
- **Edge Clearance:** A support cannot be placed closer than **2 units** from the left or right edge of the panel.
- **Cantilever Limit:** The first and last support of a continuous segment of panels cannot be further than **16 units** from the panel's respective edge.
- **Span Limit:** The distance between any two consecutive supports on a single panel cannot exceed **48 units**.

### 2. Joint Calculation

Joints are connectors used to secure adjacent panels.

- **Horizontal Connection:** A joint is required between two side-by-side panels if the horizontal distance (gap) between them is **less than 1 unit**.
- **Shared Connections (Grid Layout):** In a multi-row layout, joints can be shared between rows. If the vertical gap between rows is minimal (adjacent borders), a single joint can connect up to **4 panels** (joining the bottom corners of the upper row and the top corners of the lower row).

## Requirements & Expectations

- **Language:** English (code, comments, and naming conventions).
- **Code Quality:** Use **SOLID principles**. The code should be clean, modular, and extensible.
- **Documentation:** Public methods must have concise documentation.
- **Error Handling:** The service should handle invalid inputs gracefully.

## Expected Output

The service must return a structured result containing two distinct collections: **Mounts** and **Joints**. Each item in these collections needs to represent a precise location on the roof, clearly defined by its `x` and `y` coordinates.

- **Mounts:** Positions where the panel system attaches to the rafters.
- **Joints:** Unique center positions of the connectors between panels.

## Data for the task

Below is an example data (a list of `(x, y)` coordinates representing the **top-left corner** of each panel) that should be handled as an input. You can use it to solve the task.

```
[
  {"x": 0, "y": 0}, {"x": 45.05, "y": 0}, {"x": 90.1, "y": 0},
  {"x": 0, "y": 71.6}, {"x": 135.15, "y": 0}, {"x": 135.15, "y": 71.6},
  {"x": 0, "y": 143.2}, {"x": 45.05, "y": 143.2}, {"x": 135.15, "y": 143.2},
  {"x": 90.1, "y": 143.2}
]
```

## Submission Guidelines

- **Tech Stack:** The solution must be implemented in **Ruby** or **Python**.
- **Delivery:** Provide the source code via a **public GitHub repository**.
- **Documentation:** The repository must include a `README` file with clear instructions on:
  - How to setup and run the application.
  - How to execute the test suite to verify the results.

## Example

An example layout with mounts (black circles) and joints (gray rectangles) is placed below. Vertical lines are rafters.

